

Tentu, mari kita bedah kode Canny.cs. Jika Form1.cs adalah "Wajah" aplikasi, maka Canny.cs adalah "Otak Kanan" yang sangat spesifik tugasnya: **Mendeteksi Tepi (Edge Detection)** dengan algoritma yang canggih.

Algoritma Canny dianggap sebagai "Standard Emas" dalam deteksi tepi karena hasilnya bersih (garis tipis, minim noise, dan tersambung).

Berikut adalah penjelasannya untuk persiapan UAS Anda.

1. Gambaran Besar: Bagaimana Canny Bekerja?

Sebelum masuk ke kode, pahami dulu alur logika (Pipeline) yang ada di fungsi Apply(). Ini adalah jawaban wajib jika dosen bertanya "Jelaskan langkah-langkah Canny!".

1. **Gaussian Blur:** Gambar diburamkan sedikit untuk membuang bintik-bintik (noise). Kalau tidak diburamkan, bintik debu bisa dianggap garis tepi.
2. **Sobel Operation:** Mencari area yang warnanya berubah drastis (Gradien) dan ke mana arah perubahannya (Sudut).
3. **Non-Maximum Suppression (NMS):** Menipiskan garis yang tebal menjadi setipis 1 pixel. Hanya titik "puncak" yang diambil.
4. **Hysteresis Thresholding:** Menentukan mana garis pasti (Strong), mana garis ragu-ragu (Weak), dan menyambungkan yang ragu-ragu hanya jika menempel dengan yang pasti.

2. Penjelasan Kode Per Bagian

A. Header & Constructor

Bagian ini mengatur parameter sensitivitas.

C#

```
public class Canny
{
    // Batas bawah dan atas untuk menentukan garis 'Lemah' vs 'Kuat'
    private float LowThreshold;
    private float HighThreshold;

    public Canny(float lowThresh = 20f, float highThresh = 100f)
    {
        this.LowThreshold = lowThresh;
```

```
this.HighThreshold = highThresh;  
}
```

Poin UAS:

- HighThreshold: Jika kekuatan garis di atas angka ini, otomatis jadi **Garis Tepi**.
- LowThreshold: Jika di bawah ini, otomatis **Dibuang**. (Di antaranya disebut *Weak Edge*).

B. Fungsi Utama (Apply)

Ini adalah "Manajer" yang memanggil bawahan-bawahannya secara berurutan.

C#

```
public Bitmap Apply(Bitmap source)  
{  
    if (source == null) return null;  
  
    // Tahap 1: Hapus noise (bintik-bintik)  
    Bitmap blurred = ApplyGaussian(source);  
  
    // Tahap 2: Cari kekuatan (gradient) dan arah (angle) tepi  
    float[,] gradient;  
    float[,] angle;  
    GetSobelData(blurred, out gradient, out angle);  
  
    // Tahap 3: Tipiskan garis yang tebal  
    float[,] nms = NonMaxSuppression(gradient, angle, source.Width, source.Height);  
  
    // Tahap 4: Finalisasi (Sambung-sambungkan garis)  
    return Hysteresis(nms, source.Width, source.Height);  
}
```

C. Tahap 1: Gaussian Blur (ApplyGaussian)

Teknik pemrosesan pixel tingkat lanjut menggunakan LockBits.

C#

```
private Bitmap ApplyGaussian(Bitmap src)
{
    // ... (kode setup LockBits) ...

    // Kernel Gaussian 5x5: Matriks pembobot untuk memburamkan.
    // Angka di tengah (15) paling besar karena pixel pusat paling berpengaruh.
    double[,] kernel = { { 2, 4, 5, 4, 2 }, { 4, 9, 12, 9, 4 }, ... };
    double div = 159; // Total penjumlahan semua angka di kernel (untuk pembagi).

    // Parallel.For: Menggunakan semua inti CPU (Multi-core) agar proses cepat.
    Parallel.For(2, h - 2, y =>
    {
        for (int x = 2; x < w - 2; x++)
        {
            double sum = 0;
            // Konvolusi: Mengalikan pixel tetangga dengan kernel
            for (int ky = -2; ky <= 2; ky++)
            {
                // ... perhitungan matematika konvolusi ...
            }
            // Simpan hasil rata-rata ke pixel baru
            // ...
        }
    });
    // ...
}
```

Poin UAS:

- **Kenapa pakai LockBits & Marshal?** Karena GetPixel dan SetPixel bawaan C# sangat lambat. Untuk memproses jutaan pixel berkali-kali, kita harus mengakses memori RAM secara langsung (Direct Memory Access) agar aplikasi tidak *hang*.
- **Kenapa Parallel.For?** Operasi konvolusi itu berat. Kita membagi tugas ke semua inti prosesor komputer.

D. Tahap 2: Sobel (GetSobelData)

Mencari di mana tepi berada.

C#

```
private void GetSobelData(..., out float[,] gradient, out float[,] angle)
{
    // Kernel Sobel: Alat pendeteksi tepi Vertikal (gx) dan Horizontal (gy)
    int[,] gxK = { { -1, 0, 1 }, { -2, 0, 2 }, { -1, 0, 1 } }; // Deteksi garis tegak
    int[,] gyK = { { -1, -2, -1 }, { 0, 0, 0 }, { 1, 2, 1 } }; // Deteksi garis datar

    // ... Loop pixel ...

    // Rumus Pythagoras untuk mencari kekuatan tepi (Magnitude)
    float gVal = (float)Math.Sqrt(sumX * sumX + sumY * sumY);

    // Rumus ArcTan untuk mencari arah tepi (sudutnya kemana?)
    double theta = Math.Atan2(sumY, sumX) * 180 / Math.PI;

    // NORMALISASI: Ini trik penting!
    // Mengubah rentang angka gradient yang bisa ribuan menjadi 0-255.
    // Tujuannya agar Threshold (20-100) bekerja konsisten di gambar terang/gelap.
    if (maxGradient > 0) { ... gradient[x, y] = (gradient[x, y] / maxGradient) * 255f; }
}
```

E. Tahap 3: Non-Maximum Suppression (NonMaxSuppression)

Menipiskan garis tebal. Logikanya: "Kalau saya bukan pixel paling terang dibanding tetangga saya di arah tegak lurus, maka matikan saya".

C#

```
// Kuantisasi Sudut: Kita menyederhanakan sudut menjadi 4 arah mata angin utama.
// 0 derajat (Horizontal -)
if ((ang >= 0 && ang < 22.5) ... )
{
    q = gradient[x + 1, y]; // Cek tetangga Kanan
    r = gradient[x - 1, y]; // Cek tetangga Kiri
}
```

```
// 90 derajat (Vertikal I)
else if (ang >= 67.5 && ang < 112.5)
{
    q = gradient[x, y + 1]; // Cek tetangga Atas
    r = gradient[x, y - 1]; // Cek tetangga Bawah
}
// ... (dst untuk diagonal)

// HUKUM RIMBA:
// Jika pixel ini (gradient[x,y]) LEBIH BESAR atau SAMA DENGAN tetangganya (q & r),
// dia selamat. Kalau kalah terang, dia jadi 0 (Hitam).
if (gradient[x, y] >= q && gradient[x, y] >= r)
    nms[x, y] = gradient[x, y];
else
    nms[x, y] = 0;
```

F. Tahap 4: Hysteresis (Hysteresis) - Tahap Terakhir

Inilah yang membuat Canny spesial. Teknik ini memperbaiki garis putus-putus.

C#

```
// 1. Klasifikasi Awal
if (val >= HighThreshold) {
    // PASTI TEPI (Strong). Tandai Putih (255).
    // Masukkan koordinatnya ke Stack (Daftar Antrean) untuk dicek tetangganya nanti.
    edgeStack.Push(new Point(x, y));
}
else if (val >= LowThreshold) {
    // RAGU-RAGU (Weak). Tandai Abu-abu (50).
    result[...] = weak;
}
else {
    // SAMPAH. Buang jadi Hitam (0).
}

// 2. Flood Fill (Menyambungkan Garis)
// Selama masih ada titik 'Strong' di antrean...
while (edgeStack.Count > 0) {
    Point p = edgeStack.Pop();
```

```

// Cek 8 tetangga di sekitarnya...
if (result[nk] == weak) {
    // Jika tetangganya 'Lemah' (Abu-abu), BERARTI DIA TEMAN!
    // Ubah jadi 'Kuat' (Putih) dan masukkan ke antrian.
    result[...] = strong;
    edgeStack.Push(...);
}
}

```

```

// 3. Bersihkan Sisa
// Cari semua pixel yang masih Abu-abu (Weak) tapi tidak tersentuh proses di atas.
// Artinya dia garis lemah yang sendirian (noise). Hapus jadi Hitam.

```

Simulasi Pertanyaan UAS untuk Canny.cs

Berikut adalah pertanyaan teknis yang sering muncul terkait implementasi Canny manual seperti ini:

1. Pertanyaan tentang Urutan Proses

Dosen: "Kenapa harus di-Blur dulu? Kenapa tidak langsung Sobel saja biar cepat?"

Jawaban:

"Karena turunan pertama (Sobel) sangat sensitif terhadap noise (gangguan). Tanpa Gaussian Blur, setiap bintik debu kecil di gambar akan dianggap sebagai tepi yang tajam, sehingga hasil akhirnya akan kotor penuh bintik putih."

2. Pertanyaan tentang Performa

Dosen: "Di kode ini ada LockBits, Marshal.Copy, dan Unsafe Code. Apa bedanya dengan pakai GetPixel() biasa?"

Jawaban:

"Di Canny, kita melakukan 4 kali loop menyusuri seluruh gambar (Blur, Sobel, NMS, Hysteresis).

Kalau pakai GetPixel, setiap kali ambil warna, sistem melakukan pengecekan keamanan memori yang lambat.

Dengan LockBits, kita mengunci memori gambar di RAM dan memanipulasinya langsung sebagai array byte (byte[]), yang kecepatannya ribuan kali lebih cepat."

3. Pertanyaan tentang Logika Hysteresis

Dosen: "Jelaskan konsep 'Strong', 'Weak', dan bagaimana mereka berinteraksi di tahap Hysteresis!"

Jawaban:

"Ini konsep seleksi bertingkat, Pak.

- **Strong (Kuat):** Pixel yang nilainya di atas HighThreshold. Ini pasti tepi.
- **Weak (Lemah):** Pixel yang nilainya di antara Low & High. Ini mungkin tepi, mungkin noise.

Di tahap Hysteresis, pixel **Weak** hanya akan diselamatkan (jadi putih) jika dia **terhubung/bersentuhan** dengan pixel **Strong**. Kalau dia Weak dan sendirian (jomblo), dia dianggap noise dan dihapus."

4. Pertanyaan tentang Alpha Channel (Transparansi)

Dosen: "Saya lihat di baris akhir ada `result[k + 3] = 255;`. Itu buat apa?"

Jawaban:

"Itu adalah channel Alpha (Transparansi).

Dalam format `Format32bppArgb`, urutannya adalah Blue, Green, Red, Alpha. Kalau Alpha-nya 0, gambarnya jadi transparan (tembus pandang). Kita set ke 255 (Penuh) agar background hitam hasil deteksi tepinya terlihat pekat dan tidak bolong."

5. Pertanyaan Matematika

Dosen: "Bagaimana cara komputer tahu arah garis (sudut) di tahap Sobel?"

Jawaban:

"Menggunakan fungsi `ArcTan` (Arcus Tangen) dari rasio gradien vertikal (G_y) dibagi gradien horizontal (G_x).

Di kode, tertulis `Math.Atan2(sumY, sumX)`. Hasilnya adalah sudut dalam derajat yang memberitahu kita ke arah mana intensitas warna berubah paling drastis."